Lecture 1: Intro

1. Optimization Problem (General Form)

$$\min_x f(x) \quad \text{s.t.} \quad x \in S \subseteq \mathbb{R}^n$$

Variable: $x = (x_1, \ldots, x_n)^T$
Objective function: $f : \mathbb{R}^n \to \mathbb{R}$
Feasible set: S — points satisfying all constraints
Optimal solution: $x^* = \arg\min_{x \in S} f(x)$, where $f(x^*) \le f(x) \; \forall x \in S$
Optimal value: $f(x^*)$
max ↔ min equivalence: $\max f(x) \equiv \min -f(x)$
Constraint types (Constrained NLP)

$$S = \{x \in \mathbb{R}^n \mid g_i(x) = 0, \; i = 1, \ldots, m; \; h_j(x) \le 0, \; j = 1, \ldots, p\}$$

2. Local vs Global Minimizers

Term	Definition
Local minimizer	$\exists \varepsilon > 0$ s.t. $f(x) \ge f(x^*) \; \forall x \in S \cap B_\varepsilon(x^*)$
Strict local min	$f(x) > f(x^*) \; \forall x \in S \cap B\varepsilon(x^*) \setminus \{x^*\}$
Global minimizer	$f(x) \ge f(x^*) \; \forall x \in S$
Strict global min	$f(x) > f(x^*) \; \forall x \in S \setminus \{x^*\}$

Every global minimizer is a local minimizer. The converse is **not** generally true.

3. Gradient Vector

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right)^T$$

$-\nabla f(x^*)$: direction of **steepest descent**
$\nabla f(x^*)$: direction of **steepest ascent**
Key result: $\nabla(b^T x) = b$

4. Hessian Matrix

$$H_f(x) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}\right)_{n \times n}$$

Symmetric when f has continuous second-order derivatives.
Example: $f(x) = x_1^3 + 2x_1x_2 + x_2^2$

$$\nabla f = \begin{pmatrix} 3x_1^2 + 2x_2 \\ 2x_1 + 2x_2 \end{pmatrix}, \quad H_f = \begin{pmatrix} 6x_1 & 2 \\ 2 & 2 \end{pmatrix}$$

5. Matrix Definiteness

Type	Condition
Positive semidefinite (PSD)	$x^T A x \ge 0 \; \forall x$ — all eigenvalues $\lambda \ge 0$
Positive definite (PD)	$x^T A x > 0 \; \forall x \ne 0$ — all eigenvalues $\lambda > 0$
Negative semidefinite	all eigenvalues $\lambda \le 0$
Negative definite	all eigenvalues $\lambda < 0$
Indefinite	$\exists$ both positive and negative eigenvalues

PD ⇒ PSD, but PSD ⇏ PD.

6. Optimality Conditions (Unconstrained NLP)

A point x^* is a **stationary point** if $\nabla f(x^*) = 0$.
Necessary Conditions (if x^* is a local min):
1. $\nabla f(x^*) = 0$
2. $H_f(x^*)$ is **positive semidefinite**

Sufficient Conditions (to confirm x^* is a local min):
1. $\nabla f(x^*) = 0$
2. $H_f(x^*)$ is **positive definite**

If $H_f(x^*)$ is **not** PSD ⟹ x^* is **not** a local minimizer.

7. Convex Sets & Functions

Convex set: $x, y \in D \Rightarrow \lambda x + (1-\lambda) y \in D \; \forall \lambda \in [0,1]$
Convex function: $f(\lambda x + (1-\lambda)y) \le \lambda f(x) + (1-\lambda) f(y) \; \forall x, y \in D, \; \lambda \in [0,1]$
Strictly convex: strict inequality for distinct x, y, $\lambda \in (0,1)$
Hessian Test for Convexity (Theorem 1.25)

Hessian condition (at all $x \in D$)	Conclusion
$H_f(x)$ is PSD	f is convex
$H_f(x)$ is PD	f is **strictly** convex
$H_f(x)$ is indefinite at some $\hat{x}$	f is neither convex nor concave

Why convexity matters:
Any **local minimizer** of a convex function is a **global minimizer**.
If f is **strictly convex**, the global minimizer is **unique**.

8. Quick Reference: Calculation Steps

To find & verify a local minimizer (unconstrained):
1. Solve $\nabla f(x^*) = 0$ → find stationary points
2. Compute $H_f(x^*)$
3. Find eigenvalues of $H_f(x^*)$:
   - All > 0: local min ✓ (sufficient)
   - Any < 0: not a local min ✗ (necessary fails)

To check convexity:
1. Compute $H_f(x)$ generally
2. Check eigenvalues for all $x \in D$

Lecture 2: Gradient Descent & Linear Regression

1. Gradient Descent Basics

Stationary point: $\nabla f(x) = 0$ — necessary condition for a local min.
Key property of gradient (Proposition 1):

$\nabla f(x^*)^T d$ = rate of change of f moving from x^* in direction d
f **decreases most rapidly** along $-\nabla f(x^*)$ (steepest descent direction)
f **increases most rapidly** along $\nabla f(x^*)$

General iterative update:

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

where $\alpha_k > 0$ is the step length/learning rate and $p^{(k)}$ is the search direction.

2. Descent Direction

$p^{(k)}$ is a **descent direction** at $x^{(k)}$ if:

$$\nabla f(x^{(k)})^T p^{(k)} < 0$$

This guarantees $\exists \delta > 0$ such that $f(x^{(k)} + \alpha p^{(k)}) < f(x^{(k)})$ for all $\alpha_k \in (0, \delta)$.

Tip: $p = -\nabla f(x^{(k)})$ always satisfies this since $\nabla f^T(-\nabla f) = -\|\nabla f\|^2 \le 0$.

3. Steepest Descent Method

Update rule:

$$x^{(k+1)} = x^{(k)} - \alpha_k \nabla f(x^{(k)})$$

Stopping criterion: $\|\nabla f(x^{(k)})\| \le \varepsilon$
Step Length Options

Method	Description
Constant	Fix $\alpha_k = c$. Too small → slow; too large → diverge
Exact line search	$\alpha_k = \arg\min_{\alpha > 0} f(x^{(k)} + \alpha p^{(k)})$, solve $\phi'(\alpha) = 0$
Backtracking	Start large, shrink until Armijo condition holds

Exact Line Search
Solve: $\min_{\alpha > 0} \phi(\alpha) = f(x^{(k)} + \alpha p^{(k)})$
Set $\phi'(\alpha) = 0$ and solve for α_k.
Backtracking Line Search (Armijo)
Parameters: $\bar{\alpha} > 0$, $\rho \in (0,1)$, $c_1 \in (0,1)$ (e.g. $\bar{\alpha} = 1, \rho = 0.9, c_1 = 10^{-4}$)
Shrink $\alpha \leftarrow \rho\alpha$ until:

$$f(x^{(k)} + \alpha p^{(k)}) \le f(x^{(k)}) + c_1 \alpha \nabla f(x^{(k)})^T p^{(k)}$$

Note: RHS < LHS initially since $\nabla f^T p < 0$, so the condition requires sufficient decrease.

4. Properties of Steepest Descent (Exact Line Search)

- **Perpendicular steps:** consecutive steps are orthogonal
- **Monotone decrease:** $f(x^{(k+1)}) < f(x^{(k)})$ if $\nabla f(x^{(k)}) \ne 0$
- **Zig-zag behaviour** near the solution (inherently slow)

5. Linear Regression — One Variable

Model: $f(x) = \beta_1 x + \beta_0$
Cost function:

$$L(\beta_0, \beta_1) = \frac{1}{2} \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)^2$$

Gradients:

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)$$

$$\frac{\partial L}{\partial \beta_1} = \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i) x_i$$

Update rule:

$$\beta_0^{(k+1)} = \beta_0^{(k)} - \alpha_k \sum_{i=1}^n (\beta_1^{(k)} x_i + \beta_0^{(k)} - y_i)$$

$$\beta_1^{(k+1)} = \beta_1^{(k)} - \alpha_k \sum_{i=1}^n (\beta_1^{(k)} x_i + \beta_0^{(k)} - y_i) x_i$$

6. Linear Regression — Multiple Variables

Model: $f(x) = \beta^T x + \beta_0, \quad \beta \in \mathbb{R}^p$
Cost function:

$$L(\beta_0, \beta) = \frac{1}{2} \sum_{i=1}^n (\beta^T x_i + \beta_0 - y_i)^2$$

Gradients:

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^n (\beta^T x_i + \beta_0 - y_i)$$

$$\frac{\partial L}{\partial \beta_j} = \sum_{i=1}^n (\beta^T x_i + \beta_0 - y_i) x_{ij} \quad j = 1, \ldots, p$$

7. Normal Equation (Analytical Solution)

Define augmented matrix and vectors:

$$\tilde{X} = \begin{bmatrix} 1 & x_1^T \\ 1 & x_2^T \\ \vdots & \vdots \\ 1 & x_n^T \end{bmatrix} \in \mathbb{R}^{n \times (p+1)}, \quad \tilde{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$$

Normal equation:

$$\tilde{X}^T \tilde{X} \tilde{\beta} = \tilde{X}^T Y$$

Solution (if $\tilde{X}^T \tilde{X}$ invertible):

$$\tilde{\beta} = (\tilde{X}^T \tilde{X})^{-1} \tilde{X}^T Y$$

Case	Condition	Solutions
Over-determined	$n \gg p$, $\tilde{X}^T \tilde{X}$ invertible	Unique
Under-determined	$n < p$, $\tilde{X}^T \tilde{X}$ not invertible	Infinite

8. Feature Scaling (Standardisation)

$$X_{:j} \leftarrow \frac{X_{:j} - \text{mean}(X_{:j})}{\text{std}(X_{:j})}, \qquad Y \leftarrow \frac{Y - \text{mean}(Y)}{\text{std}(Y)}$$

9. Steepest Descent vs. Normal Equation

	Steepest Descent	Normal Equation
Type	Iterative	Analytical
Step length	Must be chosen	Not needed
Large p	Works well	Slow (matrix inversion)
When to use	$p > 5000$	$p \le 5000$

Lecture 3: Logistic Regression & Regularization

1. Logistic Regression

Sigmoid (Logistic) Function

$$g(z) = \frac{1}{1 + e^{-z}}, \quad 0 < g(z) < 1$$

- $g(0) = 0.5$, $g(z) \to 1$ as $z \to +\infty$, $g(z) \to 0$ as $z \to -\infty$

Model

$$f(x) = g(\beta^T \tilde{x}), \quad \tilde{\beta} = [\beta_0; \beta], \quad \tilde{x} = [1; x]$$

Probabilistic Interpretation

$$f(x) = P(y = 1 \mid x; \tilde{\beta}), \quad 1 - f(x) = P(y = 0 \mid x; \tilde{\beta})$$

Prediction Rule
- Predict $y = 1$ if $f(x) \ge 0.5$, i.e., $\beta^T \tilde{x} \ge 0$
- Predict $y = 0$ if $f(x) < 0.5$, i.e., $\beta^T \tilde{x} < 0$

Decision Boundary

$$\beta_0 + \beta^T x = 0$$

Features p	Boundary shape
1	Point
2	Line
3	Plane
general	$(p-1)$-dimensional subspace

2. Cost Function (Negative Log-Likelihood)

Per-sample cost

$$-y_i \log(f(x_i)) - (1 - y_i) \log(1 - f(x_i)) = \begin{cases} -\log(f(x_i)) & \text{if } y_i = 1 \\ -\log(1 - f(x_i)) & \text{if } y_i = 0 \end{cases}$$

Total cost (simplified form)

$$L(\beta_0, \beta) = \sum_{i=1}^n \log(1 + e^{\beta_0 + \beta^T x_i}) - y_i(\beta_0 + \beta^T x_i)$$

3. Gradient of the Cost Function

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^n (f(x_i) - y_i)$$

$$\frac{\partial L}{\partial \beta_j} = \sum_{i=1}^n (f(x_i) - y_i) x_{ij}, \quad j = 1, \ldots, p$$

Note: Same form as linear regression gradient, but $f(x_i)$ is the sigmoid output.

4. Multi-class: One-vs-Rest

For K classes, train K binary classifiers $f_1, \ldots, f_K$.
Predict class $k^* = \arg\max_k f_k(x)$.

5. Regularization

Ridge (L2)

$$\text{Regularizer: } \|\beta\|^2 = \lambda \sum_{j=1}^p \beta_j^2$$

- Differentiable; shrinks all β_j toward 0 but rarely to exactly 0.

Logistic + Ridge:

$$\min_{\tilde{\beta}} \sum_{i=1}^n \log(1 + e^{\beta_0 + \beta^T x_i}) - y_i(\beta_0 + \beta^T x_i) + \lambda \sum_{j=1}^p \beta_j^2$$

Linear + Ridge (Normal Equation, assuming $\beta_0 = 0$):

$$\min_\beta \frac{1}{2} \|X\beta - Y\|^2 + \lambda \|\beta\|^2$$

$$\Rightarrow \quad \beta = (2\lambda I + X^T X)^{-1} X^T Y$$

Lasso (L1)

$$\text{Regularizer: } \|\beta\|_1 = \lambda \sum_{j=1}^p |\beta_j|$$

- Non-differentiable; forces some β_j to exactly **zero** → feature selection.
- Gradient descent **not applicable**; requires specialised solvers.

Lasso problem:

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2} \|X\beta - Y\|^2 + \lambda \|\beta\|_1$$

6. Ridge vs. Lasso Summary

	Ridge	Lasso
Penalty	$\lambda \|\beta\|^2$	$\lambda \|\beta\|_1$
Differentiable	✓	✗
Shrinks β_j	Toward 0	To exactly 0
Feature selection	✗	✓
Solver	Gradient descent / Normal eq.	Specialised (e.g. proximal)

Larger λ → stronger regularization → simpler model (risk of underfitting).
Smaller λ → weaker regularization → more complex model (risk of overfitting).

Lecture 4: Proximal Gradient Method

1. Norms

Norm	Definition
ℓ_1	$\|x\|_1 = \Sigma_i \|x_i\|$
ℓ_2	$\|x\|_2 = \sqrt{(\Sigma_i x_i^2)}$
ℓ_∞	$\|x\|_\infty = \max_i \|x_i\|$
ℓp	$\|x\|p = (\Sigma_i \|x_i\|^p)^{\wedge}(1/p)$
Frobenius	$\|A\|^2_F = \langle A, A \rangle = \text{Tr}(A^T A)$

Inner product (matrices): $\langle A, B \rangle = \text{Tr}(A^T B) = \Sigma_i \Sigma_j A_{ij} B_{ij}$
Inner product (vectors): $\langle x, y \rangle = x^T y = \Sigma_j x_j y_j$

2. Projection onto Closed Convex Set C

```
ΠC(z) = argmin  ½‖x − z‖²
          x ∈ C
```

Characterisation: $x^* = \Pi C(z) \iff \langle z - x^*, x - x^* \rangle \le 0$ for all $x \in C$

Set C	ΠC(z)
$\mathbb{R}^n_+$ (positive orthant)	max(z, 0) (elementwise)
ℓ_2-ball { $\|x\|_2 \le 1$ }	$z / \max(\|z\|_2, 1)$
S^n_+ (PSD cone), $A = Q \Lambda Q^T$	$Q \cdot \text{diag}(\max(\lambda_i, 0)) \cdot Q^T$

3. Normal Cone

```
NC(x) = { z  |  ⟨z, x − x⟩ ≤ 0  for all x ∈ C }
```

Key equivalence: $u \in NC(y) \iff y = \Pi C(y + u)$
Property: If $x \in \text{int}(C)$, then $NC(x) = \{0\}$
Example — C = [0, 1]:

x	NC(x)
0	(−∞, 0]
1	[0, +∞)
(0, 1)	{0}
$x \notin C$	∅

Example — C = { x ∈ ℝ² : ‖x‖₂ ≤ 1 }:

x	NC(x)
‖x‖ = 1	{ λx : λ ≥ 0 }
‖x‖ < 1	{0}

4. Subdifferential

```
∂f(x) = { v  |  f(z) ≥ f(x) + ⟨v, z − x⟩  for all z }
```

- If f differentiable at x: **∂f(x) = {∇f(x)}**
- **Global optimality:** x is a global minimiser ⟺ **0 ∈ ∂f(x)**
- **Indicator function:** ∂δC(x) = NC(x) for x ∈ C

Example — f(x) = |x|:

x	∂f(x)
x < 0	{−1}
x = 0	[−1, 1]
x > 0	{1}

Lasso sparsity condition — at optimum β:

```
βᵢ < 0  ⟺  [X^(Xβ − Y)]ᵢ =  λ
βᵢ > 0  ⟺  [X^(Xβ − Y)]ᵢ = −λ
βᵢ = 0  ⟺  |[X^(Xβ − Y)]ᵢ| ≤ λ   ← sparsity!
```

5. Fenchel Conjugate

```
f*(y) = sup { ⟨y, x⟩ − f(x) }
        x
```

f* is always convex and closed (even if f is not).
If f is closed proper convex, then (f*)* = f.
Key triple equivalence:

```
f(x) + f*(y) = ⟨x, y⟩  ⟺  y ∈ ∂f(x)  ⟺  x ∈ ∂f*(y)
```

Examples:

f(x)	f*(y)
‖x‖₁	δC(y), C = { y : ‖y‖∞ ≤ 1 }
δC(x) (indicator)	sup{ ⟨y, x⟩ : x ∈ C } (support function)

6. Moreau Envelope & Proximal Operator

```
Pf(x) = argmin { f(y) + ½‖y − x‖² }    ← proximal mapping
        y

Mf(x) =  min  { f(y) + ½‖y − x‖² }    ← Moreau envelope
         y
```

Properties:
- ∇Mf(x) = x − Pf(x) (Mf is always differentiable)
- argmin f = argmin Mf
- PS_C(x) = ΠC(x)

Moreau Decomposition:

```
x       = Pf(x) + Pf*(x)
½‖x‖²   = Mf(x) + Mf*(x)
```

Soft Thresholding — prox of f(x) = λ|x|:

```
Pf(x) = Sλ(x) = sign(x) · max{ |x| − λ, 0 }
```

Applied elementwise to x = [x₁; …; xₙ]:

```
[Sλ(x)]ᵢ = sign(xᵢ) · max{ |xᵢ| − λ, 0 }
```

Huber function (Moreau envelope of f(x) = λ|x|):

```
Mf(x) = ½x²            if |x| ≤ λ
        λ|x| − λ²/2    if |x| > λ
```

7. Proximal Gradient (PG) Method

Problem: min f(β) + g(β), where f is smooth and g is convex non-smooth.
Key insight — gradient step on f only, then prox on g:

```
β^(k+1) = Pαg( β^(k) − α∇f(β^(k)) )
```

Algorithm:

```
choose β^(0),  step size α > 0
repeat:
    β^(k+1) = Pαg( β^(k) − α∇f(β^(k)) )
until convergence
```

Convergence: f(β^(k)) + g(β^(k)) − optimal ≤ O(1/k)

8. Accelerated Proximal Gradient (APG) Method

Algorithm (FISTA-style):

```
choose β^(0),  step size α > 0,  t₀ = t₁ = 1
repeat:
    β^(k)   = β^(k) + (tₖ − 1)/tₖ₊₁ · (β^(k) − β^(k−1))   ← momentum
    β^(k+1) = Pαg( β^(k) − α∇f(β^(k)) )
    tₖ₊₁ = ( 1 + √(1 + 4tₖ²) ) / 2
until convergence
```

Convergence: f(β^(k)) + g(β^(k)) − optimal ≤ O(1/k²)
Step size rule: α ∈ (0, 1/L), where L = Lipschitz constant of ∇f

9. APG Applied to Lasso

```
min  ½‖Xβ − Y‖²  +  λ‖β‖₁
 β
```

- ∇f(β) = X^T(Xβ − Y)
- Lipschitz constant: L = λ_max(X^TX)
- Step size: α = 1/L

Iteration:

```
β^(k)   = β^(k) + (tₖ − 1)/tₖ₊₁ · (β^(k) − β^(k−1))

β^(k+1) = Sα/L(L)( β^(k) − (1/L) · X^T(Xβ^(k) − Y) )
```

Optimality condition: β* is optimal ⟺ β* = Pg(β* − ∇f(β*))
Stopping criterion (tolerance ε > 0):

```
‖ β^(k) − Sλ( β^(k) − X^T(Xβ^(k) − Y) ) ‖ < ε
```

10. Complexity Summary

Method	Convergence rate	Iterations to 10^{-6} error
PG	O(1/k)	~O(10^6)
APG	O(1/k²)	~O(10^3)

- APG has the **same per-iteration cost** as PG (one prox + one gradient eval)
- **Restart trick:** rerun APG every 100–200 iterations from the latest iterate

Lecture 5: SVM

1. SVM Basics

Setup: Data (x_i, y_i) where $x_i \in \mathbb{R}^p$, $y_i \in \{-1, 1\}$, classes assumed **linearly separable**.
Classifier: $f(x) = \text{sign}(\beta^T x + \beta_0)$
Distance of point x to hyperplane $H = \{\beta^T x + \beta_0 = 0\}$:

$$d = \frac{|\beta^T x + \beta_0|}{\|\beta\|}$$

Margin (distance to closest point from hyperplane):

$$\gamma = \min_{i=1,\ldots,n} \frac{|\beta^T x_i + \beta_0|}{\|\beta\|}$$

2. SVM Primal Problem

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \quad \text{s.t.} \quad y_i(\beta^T x_i + \beta_0) \ge 1, \quad \forall i \in [n]$$

- Margin $= \frac{1}{\|\beta\|}$ at optimum
- **Support vectors**: points x_i where $y_i(\beta^T x_i + \beta_0) = 1$ (tight constraint)

3. KKT Conditions (General)

For primal $\min f(x)$ s.t. $g_i(x) = 0$, $h_j(x) \le 0$, the **KKT conditions** at a regular point x^* are:

Condition	Equation
Stationarity	$\nabla f(x^*) + \sum v_i^* \nabla g_i(x^*) + \sum u_j^* \nabla h_j(x^*) = 0$
Primal feasibility	$g_i(x^*) = 0, \; h_j(x^*) \le 0$
Dual feasibility	$u_j^* \ge 0$
Complementary slackness	$u_j^* h_j(x^*) = 0$

Lagrangian: $L(x, v, u) = f(x) + \sum v_i g_i(x) + \sum u_j h_j(x)$
Dual function (always concave): $\theta(v, u) = \min_x L(x, v, u)$
Weak duality: $\theta(v, u) \le f(x)$ for any dual-feasible (v, u) and primal-feasible x
Strong duality holds under: convex f, h_j; affine g_i; and **Slater's condition** ($\exists \tilde{x}$: $g_i(\tilde{x}) = 0, \; h_j(\tilde{x}) < 0$).

4. SVM Dual Problem

Lagrangian (with $\alpha \in \mathbb{R}^n_+$):

$$L(\beta, \beta_0, \alpha) = \frac{1}{2} \|\beta\|^2 + \sum_{i=1}^n \alpha_i (1 - y_i(\beta^T x_i + \beta_0))$$

Setting $\frac{\partial L}{\partial \beta} = 0$ and $\frac{\partial L}{\partial \beta_0} = 0$ gives:

$$\beta^* = \sum_{i=1}^n \alpha_i y_i x_i \qquad \sum_{i=1}^n \alpha_i y_i = 0$$

Dual problem:

$$\max_\alpha \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0, \; \alpha_i \ge 0$$

Recovering primal from dual:

$$\beta^* = \sum_{i=1}^n \alpha_i^* y_i x_i \qquad \beta_0^* = y_k - \sum_{i=1}^n \alpha_i^* y_i \langle x_i, x_k \rangle \text{ for any } k \text{ with } \alpha_k^* > 0$$

Complementary slackness: $\alpha_i^* > 0 \Rightarrow y_i(\beta^*)^T x_i + \beta_0^*) = 1$ (i.e., x_i is a support vector)
Decision boundary (test point x):

$$f(x) = \text{sign}\left(\sum_{i: \alpha_i^* > 0} \alpha_i^* y_i \langle x_i, x \rangle + \beta_0^*\right)$$

5. Kernels

Replace $\langle x_i, x_j \rangle \to K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ in the dual.

Kernel	Formula
Linear	$K(a, b) = a^T b$
Homogeneous polynomial degree d	$K(a, b) = (a^T b)^d$
Inhomogeneous polynomial degree d	$K(a, b) = (a^T b + 1)^d$
Gaussian (RBF)	$K(a, b) = \exp\left(-\frac{\|a - b\|^2}{2\sigma^2}\right)$

Key point: Computing $K(a, b)$ is $O(p)$; computing $\phi(a)$ explicitly can be $O(p^2)$ or higher.

6. Soft-Margin SVM

When classes are **not separable**, introduce slack $\varepsilon_i \ge 0$:

$$\min_{\beta, \beta_0, \varepsilon} \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^n \varepsilon_i \quad \text{s.t.} \quad y_i(\beta^T x_i + \beta_0) \ge 1 - \varepsilon_i, \; \varepsilon_i \ge 0$$

Equivalent unconstrained form (**hinge loss**):

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^n \max\{1 - y_i(\beta^T x_i + \beta_0), 0\}$$

Soft-margin dual:

$$\max_\alpha \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0, \; 0 \le \alpha_i \le C$$

C controls the margin–violation trade-off. Large C = hard margin.

7. SVM vs. Logistic Regression

	SVM (soft)	Logistic Regression
Loss	$\max(1 - z, 0)$ (hinge)	$\log(1 + e^{-z})$ (logistic)
Regularization	$\frac{1}{2} \|\beta\|^2$	$\lambda \|\beta\|^2$
Target	$z = y_i(\beta^T x_i + \beta_0) \ge 1$	$z \gg 0$

Logistic loss is a smooth approximation to hinge loss.

Quick Reference: Deriving a Dual

1. Write the **Lagrangian** L with multipliers (≥ 0 for ≤ 0 constraints)
2. Set $\nabla_x L = 0$ to find x^* in terms of multipliers
3. Substitute back to get **dual function** θ
4. Maximize θ subject to multiplier constraints → **dual problem**

Lecture 6: Block Coordinate Descent (BCD)

Target Problem

$$\min_{x \in \mathbb{R}^n} F(x) = f(x) + \sum_{i=1}^m r_i(x_i)$$

- f: convex, differentiable (smooth part)
- $r_i(x_i)$: closed proper convex, **separable** (non-smooth part)

Coordinate-wise Minimizer

Definition: $\bar{x}$ is a coordinate-wise minimizer if $\bar{x} \in \text{dom} f$ and:

$$f(\bar{x} + d e_i) \ge f(\bar{x}) \quad \forall i \in [n], \; d \in \mathbb{R}$$

Equivalently, for each i:

$$\bar{x}_i \in \arg\min_{x_i} f(\bar{x}_1, \ldots, \bar{x}_{i-1}, x_i, \bar{x}_{i+1}, \ldots, \bar{x}_n)$$

When does coordinate-wise min ⟹ global min?

Condition	Result
f differentiable at $\bar{x}$	✓ coord-wise min ⟹ global min (since $\nabla_i f(\bar{x}) = 0 \; \forall i \Rightarrow \nabla f(\bar{x}) = 0$)
f non-differentiable, non-separable	✗ not guaranteed
f non-differentiable, **separable** $r_i(x_i)$	✓ coord-wise min ⟹ global min

Coordinate Descent Algorithm

Cycle through $i = 1, 2, \ldots, n$ and solve one 1D subproblem at a time:

$$x_i^{(k+1)} \leftarrow \arg\min_{x_i} f(x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}, x_i, x_{i+1}^{(k)}, \ldots, x_n^{(k)}) + r_i(x_i)$$

Use updated values $x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}$ immediately. **Cannot be parallelised.**

Key Tools for Derivations

Optimality condition (convex f):

$$x^* \in \arg\min f(x) \iff 0 \in \partial f(x^*)$$

Proximal mapping:

$$P_{\alpha g}(x) = \arg\min_y \left\{ g(y) + \frac{1}{2} \|y - x\|^2 \right\}$$

Characterisation:

$$y = P_g(x) \iff x \in y + \partial g(y)$$

Special cases:

$$P_{\lambda|\cdot|}(x) = S_\lambda(x) \qquad P_{\delta_C}(x) = \Pi_C(x)$$

Soft-thresholding operator:

$$S_\lambda(t) = \text{sign}(t) \cdot \max\{|t| - \lambda, 0\}$$

Subdifferential of $|\cdot|$ at 0:

$$\partial|0| = [-1, 1]$$

Application 1: Linear Regression

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2} \|X\beta - Y\|^2$$

Update rule:

$$\beta_i \leftarrow \frac{X_{:i}^T (Y - X_{:-i}\beta_{-i})}{\|X_{:i}\|^2} = \beta_i - \frac{X_{:i}^T (X\beta - Y)}{\|X_{:i}\|^2}$$

Parameter-free (no step size). $X_{:-i} = X$ with i-th column removed; $\beta_{-i} = \beta$ with i-th entry removed.

Application 2: Lasso

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2} \|X\beta - Y\|^2 + \lambda \|\beta\|_1, \quad \lambda > 0$$

Non-differentiable term is separable: $\lambda \|\beta\|_1 = \sum_{i=1}^p \lambda |\beta_i|$
Update rule:

$$\beta_i \leftarrow S_{\lambda / \|X_{:i}\|^2}\left(\frac{X_{:i}^T (Y - X_{:-i}\beta_{-i})}{\|X_{:i}\|^2}\right)$$

Same inner quantity as linear regression, wrapped in soft-thresholding. Produces sparse solutions.

Application 3: Box-Constrained Regression

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2} \|X\beta - Y\|^2 \quad \text{s.t.} \quad l \le \beta \le u$$

Indicator $\delta_C(\beta) = \sum_i \delta_{C_i}(\beta_i)$ is separable.
Update rule:

$$\beta_i \leftarrow \Pi_{C_i}\left(\frac{X_{:i}^T (Y - X_{:-i}\beta_{-i})}{\|X_{:i}\|^2}\right)$$

Clipping (projection onto $[l_i, u_i]$):

$$\Pi_{C_i}(t) = \begin{cases} u_i & t > u_i \\ t & l_i \le t \le u_i \\ l_i & t < l_i \end{cases}$$

Application 4: SVM — SMO

BCD in **blocks of 2**. Select pair (α_i, α_j) violating KKT, then solve:

$$\min_{\alpha_1, \alpha_2} \frac{1}{2} \alpha_1^2 K_{11} \alpha_1^2 + \frac{1}{2} \alpha_2^2 K_{22} \alpha_2^2 + y_1 y_2 K_{12} \alpha_1 \alpha_2 - \alpha_1 - \alpha_2$$
$$\text{s.t.} \quad y_1 \alpha_1 + y_2 \alpha_2 = C, \quad 0 \le \alpha_1 \le C, \quad 0 \le \alpha_2 \le C$$

Reduces to a 1D quadratic over an interval → closed-form solution. Pairs are required to maintain $\sum_i \alpha_i y_i = 0$.

KKT (complementary slackness) conditions:

α_i	Margin condition
$\alpha_i = 0$	$y_i(\beta^T x_i + \beta_0) \ge 1$
$0 < \alpha_i < C$	$y_i(\beta^T x_i + \beta_0) = 1$
$\alpha_i = C$	$y_i(\beta^T x_i + \beta_0) \le 1$

Application 5: Parallel Projections

Find $x \in C_1 \cap C_2 \cap \cdots \cap C_m$ via:

$$\min_{x, y_i} \frac{1}{2} \sum_{i=1}^m \|y_i - x\|^2 \quad \text{s.t.} \quad y_i \in C_i$$

BCD updates (repeats until convergence):

$$y_i \leftarrow \Pi_{C_i}(x) \quad \text{for } i = 1, \ldots, m \qquad \text{(parallelisable)}$$

$$x \leftarrow \frac{1}{m} \sum_{i=1}^m y_i$$

Summary: CD vs Gradient Descent

	Coordinate Descent	Gradient Descent
Step size	Not needed	Required (α)
Per-iteration cost	1D subproblem	Full gradient ∇f
Parallelisable	✗ No	✓ Yes
Typical convergence	Fewer iterations	More iterations (fixed α)

Lecture 7: NMF

Core Problem

$$\min_{W, H \ge 0} \frac{1}{2} \|V - WH\|_F^2 \quad \text{s.t.} \quad W \ge 0, \; H \ge 0$$

Matrix	Dimensions	Role
V	m × n	Data (columns = data points)
W	m × r	Basis vectors
H	r × n	Coordinates/coefficients

Bi-convex (non-convex jointly, but convex in W or H separately).

Variants

Variant	Extra Constraint	Use Case
Standard NMF	—	General
Orthogonal NMF	$HH^T = I_r$	Hard clustering
Symmetric NMF	$H = W^T$, so $V \approx WW^T$	Community detection
Sparse NMF	$+\lambda_W \|W\|_1 + \lambda_H \|H\|_1$	Localized features

Key lemma (Orthogonal NMF): $HH^T = I_r \Rightarrow$ each column of H has **at most one positive entry**.

Algorithms

BCD / HALS
Update columns of W, then rows of H cyclically:

$$W_{:i} \leftarrow \Pi_+\left(\frac{(V - W_{:(-i)} H_{(-i):}) H_{i:}^T}{\|H_{i:}\|^2}\right), \qquad H_{i:} \leftarrow \Pi_+\left(\frac{W_{:i}^T (V - W_{:(-i)} H_{(-i):})}{\|W_{:i}\|^2}\right)$$

Proximal Gradient (PG)

$$W^{(k+1)} = \Pi_+\left(W^{(k)} + \alpha_k R^{(k)} (H^{(k)})^T\right), \qquad H^{(k+1)} = \Pi_+\left(H^{(k)} + \alpha_k (W^{(k)})^T R^{(k)}\right)$$

where $R^{(k)} = V - W^{(k)} H^{(k)}$.

Item	Formula
Frobenius norm	$\|A\|_F^2 = \mathrm{Tr}(A^TA) = \sum_{i,j} A_{ij}^2$
ℓ_1 norm	$\|A\|_1 = \sum_{i,j} \|A_{ij}\|$
Gradient w.r.t. W	$\nabla_W f = -(V - WH)H^T$
Gradient w.r.t. H	$\nabla_H f = -W^T(V - WH)$
Projection	$\Pi_+(x) = \max(x, 0)$ element-wise
Rank inequality	$\mathrm{rank}(V) \le \mathrm{rank}_+(V) \le \min(m, n)$

Cluster Membership Rules

Orthogonal / Standard NMF: point $j \in$ cluster k if $H_{kj} > H_{ij}\ \forall i \neq k$

Symmetric NMF: node $j \in$ community k if $W_{jk} > W_{ji}\ \forall i \neq k$

Lecture 8: Recommendation Systems & Matrix Completion

1. Utility Matrix and Core Concepts

Definition

Utility Matrix R: m × n matrix where R_ij = rating of user i on item j
Ω: Index set of known entries: Ω = {(i,j) | R_ij is known}
Sparsity: Most entries are unknown (blanks)

Goal
Predict missing ratings (blank entries) in the utility matrix

2. Collaborative Filtering (CF)

A. User-User CF

Step 1: Find Similar Users For user x, identify neighborhood N_x using similarity metrics:

Similarity Measures:

Jaccard Similarity (for binary matrices)

$$\mathrm{Sim}(x, y) = \frac{|S_x \cap S_y|}{|S_x \cup S_y|} \in [0, 1]$$

Where S_x = items rated by user x

Cosine Similarity

$$\mathrm{Sim}(x, y) = \frac{r_x^T r_y}{\|r_x\| \cdot \|r_y\|} \in [0, 1]$$

(Treat blanks as 0)

Normalized Cosine Similarity

$$\mathrm{Sim}(x, y) = \frac{(r_x - \bar{r}_x)^T (r_y - \bar{r}_y)}{\|r_x - \bar{r}_x\| \cdot \|r_y - \bar{r}_y\|} \in [-1, 1]$$

Where $\bar{r}_x$ = average rating of user x

Step 2: Predict Rating
For user x's rating on item i:

Naive Average:

$$r_{xi} = \frac{1}{|N(x,i)|} \sum_{y \in N(x,i)} r_{yi}$$

Weighted Average:

$$r_{xi} = \frac{\sum_{y \in N(x,i)} \mathrm{Sim}(x, y) \cdot r_{yi}}{\sum_{y \in N(x,i)} \mathrm{Sim}(x, y)}$$

Where N(x,i) = {y ∈ N_x | r_yi exists}

B. Item-Item CF
Similar to user-user, but find similar items instead:

$$r_{xi} = \frac{\sum_{j \in N(i,x)} \mathrm{Sim}(i, j) \cdot r_{xj}}{\sum_{j \in N(i,x)} \mathrm{Sim}(i, j)}$$

Where N(i,x) = {j ∈ N_i | r_xj exists}

3. Latent Factor Model

Concept
Assume the utility matrix has low-rank structure:

$$R \approx WH$$

Where:
W ∈ ℝ^(m×k): User-factor matrix (m users, k latent factors)
H ∈ ℝ^(k×n): Item-factor matrix (k latent factors, n items)
k: Number of latent factors (usually much smaller than m and n)

Prediction

$$R_{ij} = W_{i:}H_{:j} = \sum_{t=1}^{k} W_{it}H_{tj}$$

Optimization Problem

$$\min_{W,H} F(W, H) := \frac{1}{2} \sum_{(i,j) \in \Omega} (R_{ij} - W_{i:}H_{:j})^2 + \frac{\lambda}{2}(\|W\|_F^2 + \|H\|_F^2)$$

Where λ ≥ 0 is ridge regularization parameter

Gradient Descent

$$W^{(k+1)} \leftarrow W^{(k)} - \alpha_k \nabla_W F(W^{(k)}, H^{(k)})$$
$$H^{(k+1)} \leftarrow H^{(k)} - \alpha_k \nabla_H F(W^{(k)}, H^{(k)})$$

Gradients

$$[\nabla_W F]_{it} = -\sum_{j:(i,j) \in \Omega} (R_{ij} - W_{i:}H_{:j})H_{tj} + \lambda W_{it}$$
$$[\nabla_H F]_{tj} = -\sum_{i:(i,j) \in \Omega} (R_{ij} - W_{i:}H_{:j})W_{it} + \lambda H_{tj}$$

Stochastic Gradient Descent
Evaluate gradients on single ratings instead of all, faster per step but needs more iterations

4. Matrix Completion

Problem Definition
Given partially observed matrix M with known entries in Ω, recover the complete matrix:

$$\min_X \mathrm{rank}(X) \quad \text{s.t.} \quad X_{ij} = M_{ij}, \forall (i,j) \in \Omega$$

This is NP-hard!

Convex Relaxation using Nuclear Norm
Since rank minimization is **NP-hard** (rank is non-convex), we replace it with its best convex approximation — the nuclear norm:

$$\min_X \mathrm{rank}(X) \quad \Rightarrow \quad \min_X \|X\|_* \quad \text{s.t.} \quad X_{ij} = M_{ij} \qquad \text{s.t.} \quad X_{ij} = M_{ij}$$

Why it works: Minimizing $\|X\|_* = \sum_i \sigma_i$ acts as an L1 penalty on singular values, pushing many to zero and reducing rank — analogous to how L1 promotes sparsity in vectors.
Theoretical backing: Nuclear norm is the **convex envelope** of rank on $\{X \mid \sigma_1(X) \le 1\}$ — the tightest possible convex approximation. Solved efficiently via SDP (Algorithm 1) or Proximal Gradient (Algorithm 2).

5. Rank & Nuclear Norm

Rank Definition
For X ∈ ℝ^(m×n), rank(X) is:
Dimension of column space
Dimension of row space
Smallest k such that X = WH (W ∈ ℝ^(m×k), H ∈ ℝ^(k×n))
Number of non-zero singular values

Singular Value Decomposition (SVD)

$$X = U\Sigma V^T$$

Where:
U ∈ ℝ^(m×m): Orthogonal, columns are left singular vectors
Σ ∈ ℝ^(m×n): Diagonal, entries σ_i are singular values (σ_1 ≥ σ_2 ≥ ... ≥ σ_k ≥ 0)
V ∈ ℝ^(n×n): Orthogonal, columns are right singular vectors

Compact form: $X = \sum_{t=1}^{k} \sigma_t u_t v_t^T$

Nuclear Norm Definition

$$\|X\|_* = \sum_{i=1}^{\min(m,n)} \sigma_i(X)$$

(Sum of all singular values)
Properties
- **Convex** (is a norm)
- **Non-convex counterpart**: rank(X) is non-convex
- **Convex envelope**: Nuclear norm is the convex envelope of rank on {X | σ_1(X) ≤ 1}

Convex Relaxation
Replace rank minimization with nuclear norm minimization:

$$\min_X \|X\|_* \quad \text{s.t.} \quad X_{ij} = M_{ij}, \forall (i,j) \in \Omega$$

6. Recovery Guarantee (Informal)

Theorem: Under certain assumptions, if:
- True matrix rank = k
- Observed entries uniformly at random
- Number of observations ≥ C·n·k·log²(n)

Then nuclear norm minimization recovers the true matrix with high probability.

7. Semidefinite Program (SDP)

Standard SDP Form

$$\min_{X \in S^n} \langle C, X \rangle \quad \text{s.t.} \quad \langle A_i, X \rangle = b_i, \quad i \in [m] \qquad X \succ 0$$

Where:
- **C, A_i** ∈ S^n (symmetric matrices)
- **b** ∈ ℝ^m
- **X ≻ 0** means X is positive semidefinite

SDP Dual

$$\max_{y \in \mathbb{R}^m} \langle b, y \rangle \quad \text{s.t.} \quad \sum_{i=1}^{m} y_i A_i \preceq C$$

Weak Duality
For any primal feasible X and dual feasible y:

$$\langle C, X \rangle \ge \langle b, y \rangle$$

Strong Duality
If both primal and dual are strictly feasible, optimal values match.

8. ALGORITHM 1: SDP-based Matrix Completion

Nuclear Norm as SDP

$$\|X\|_* = \min_{W_1, W_2} \frac{1}{2}(\mathrm{Tr}(W_1) + \mathrm{Tr}(W_2)) \quad \text{s.t.} \quad \begin{bmatrix} W_1 & X \\ X^T & W_2 \end{bmatrix} \succeq 0$$

Matrix Completion SDP

$$\min_{W_1, W_2, X} \frac{1}{2}(\mathrm{Tr}(W_1) + \mathrm{Tr}(W_2)) \quad \text{s.t.} \quad X_{ij} = M_{ij}, \quad (i,j) \in \Omega \qquad \begin{bmatrix} W_1 & X \\ X^T & W_2 \end{bmatrix} \succeq 0$$

Can be solved by SDP solvers (SDPT3, Mosek, CVX)

9. ALGORITHM 2: PROXIMAL GRADIENT (PG)

Penalized Problem

$$\min_X \|X\|_* + \frac{1}{2\mu} \sum_{(i,j) \in \Omega} (X_{ij} - M_{ij})^2$$

Can rewrite as:

$$\min_X \underbrace{\frac{1}{2} \sum_{(i,j) \in \Omega} (X_{ij} - M_{ij})^2}_{f(X)\ \text{smooth}} + \underbrace{\mu \|X\|_*}_{g(X)\ \text{nonsmooth}}$$

Gradient of f(X)

$$[\nabla f(X)]_{ij} = \begin{cases} X_{ij} - M_{ij} & \text{if } (i,j) \in \Omega \\ 0 & \text{otherwise} \end{cases}$$

Proximal Mapping of Nuclear Norm

$$P_{\mu\|\cdot\|_*}(Y) = \arg\min_X \left\{ \mu \|X\|_* + \frac{1}{2}\|X - Y\|_F^2 \right\}$$

Computed via soft-thresholding of singular values:
1. Compute SVD: $Y = U\mathrm{Diag}(\sigma)V^T$
2. Soft-threshold: $\gamma_i = \max(\sigma_i - \mu, 0)$
3. Result: $P_{\mu\|\cdot\|_*}(Y) = U\mathrm{Diag}(\gamma)V^T$

PG Algorithm

```
Input: M, μ, step size α
Initialize: X = 0
For t = 1, 2, ...:
  G = ∇f(X)
  X = P_{μ||·||_*}(X - α·G)
Output: X
```

Note: Each iteration requires SVD, which is expensive for large matrices.

10. Netflix Prize model

Dataset
- **Training**: 100,480,507 ratings from 480,189 users on 17,770 movies
- **Testing**: 2,817,131 ratings (hidden)

Evaluation Metric

$$\mathrm{RMSE} = \sqrt{\sum_{(i,j) \in \mathrm{Test}} (R_{ij} - R_{ij}^*)^2}$$

Baselines
- **Trivial** (average): RMSE = 1.0540
- **Netflix Cinematch**: RMSE = 0.9514 (10% improvement)
- **Grand Prize**: RMSE ≤ 0.8563 (10% improvement over Cinematch)

11. KEY TAKEAWAYS

Method	Pros	Cons
User-User CF	Simple, interpretable	Doesn't scale well, new user problem
Item-Item CF	Works well in practice	Requires similar items to exist
Latent Factor	Scalable, handles cold start better	Needs optimization, non-convex
Matrix Completion (NucNorm)	Theoretical guarantees, convex	Computationally expensive (SDP/PG)

12. QUICK REFERENCE: FORMULAS FOR CALCULATIONS

Jaccard: $\frac{|A \cap B|}{|A \cup B|}$

Cosine: $\frac{a \cdot b}{\|a\| \|b\|}$

Normalized Cosine: $\frac{(a - \bar{a}) \cdot (b - \bar{b})}{\|a - \bar{a}\| \|b - \bar{b}\|}$

Weighted Average: $\frac{\sum w_i x_i}{\sum w_i}$

SVD Reconstruction: $X = U\Sigma V^T = \sum_i \sigma_i u_i v_i^T$

Nuclear Norm: $\|X\|_* = \sum_i \sigma_i(X)$

Soft Threshold: $\max(\sigma - \mu, 0)$

Lecture 9: PCA, Robust PCA & ADMM

1. PCA

Goal: Find direction w with $\|w\| = 1$ of greatest variance in data $X \in \mathbb{R}^{n \times p}$.
Covariance matrix:

$$\Sigma = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^T = \frac{1}{n} X^T X \quad \text{(assuming zero-mean data)}$$

Optimization problem:

$$\max_w w^T X^T X w \quad \text{s.t.} \quad \|w\| = 1$$

Equivalent to maximizing the **Rayleigh quotient**:

$$\max_w \frac{w^T X^T X w}{w^T w}$$

Solution: w = eigenvector of X^TX with the **largest eigenvalue** λ_1.
Steps to find the principal subspace:
1. Compute $\Sigma = \frac{1}{m}\sum_{i=1}^{n} x_i x_i^T$
2. Find eigenvalues via $\det(\Sigma - \lambda I) = 0$
3. Find eigenvector w for the largest λ via $(\Sigma - \lambda I)w = 0$

k-th component: eigenvector of X^TX with the k-th largest eigenvalue.
Equivalent low-rank formulation:

$$\min_{L: \mathrm{rank}(L) = r} \|X - L\|_F$$

⚠ PCA is **sensitive to outliers** — a few corrupted entries can distort the principal components.

2. Robust PCA

Model: $X = L_0 + S_0$, where L_0 is low-rank and S_0 is sparse.
Non-convex (NP-hard) formulation:

$$\min_{L,S} \mathrm{rank}(L) + \lambda \|S\|_0 \quad \text{s.t.} \quad L + S = X$$

Convex relaxation:

$$\min_{L,S} \|L\|_* + \lambda \|S\|_1 \quad \text{s.t.} \quad L + S = X$$

Term	Meaning	Relaxes
$\|L\|_*$	Nuclear norm = sum of singular values	$\mathrm{rank}(L)$
$\|S\|_1$	Sum of absolute values of all entries	$\|S\|_0$

Recommended λ:

$$\lambda = \frac{1}{\sqrt{\max(m, n)}}$$

3. ADMM

Target problem (2-block separable):

$$\min_{y,z} f(y) + g(z) \quad \text{s.t.} \quad Ay + Bz = c$$

Augmented Lagrangian ($\sigma > 0$):

$$\mathcal{L}_\sigma(y, z, x) = f(y) + g(z) + \langle x, Ay + Bz - c \rangle + \frac{\sigma}{2}\|Ay + Bz - c\|^2$$

Using completing the square, equivalently:

$$\mathcal{L}_\sigma(y, z, x) = f(y) + g(z) + \frac{\sigma}{2}\|Ay + Bz - c + \sigma^{-1}x\|^2 - \frac{1}{2\sigma}\|x\|^2$$

ADMM iterations:

$$y^{(k+1)} \leftarrow \arg\min_y \mathcal{L}_\sigma\left(y, z^{(k)}, x^{(k)}\right)$$
$$z^{(k+1)} \leftarrow \arg\min_z \mathcal{L}_\sigma\left(y^{(k+1)}, z, x^{(k)}\right)$$
$$x^{(k+1)} \leftarrow x^{(k)} + \tau\sigma\left(Ay^{(k+1)} + Bz^{(k+1)} - c\right)$$

Parameters: $\sigma > 0, \quad 0 < \tau < \frac{1 + \sqrt{5}}{2} \approx 1.618$
Key completing-the-square identity:

$$\langle X, Y \rangle + \frac{\sigma}{2}\|Y\|^2 = \frac{\sigma}{2}\left\|Y + \sigma^{-1}X\right\|^2 - \frac{1}{2\sigma}\|X\|^2$$

Proximal operator:

$$\mathbf{P}_{\frac{1}{\sigma}f}(v) = \arg\min_y \left\{ \frac{1}{\sigma}f(y) + \frac{1}{2}\|y - v\|^2 \right\}$$

4. ADMM for Robust PCA

Problem: $\min_{L,S} \|L\|_* + \lambda\|S\|_1$ s.t. $L + S = X \quad \Rightarrow \quad A = I,\ B = I,\ c = X$
Augmented Lagrangian:

$$\mathcal{L}\sigma(L, S, Z) = \|L\|_* + \lambda\|S\|_1 + \frac{\sigma}{2}\left\|L + S - X + \sigma^{-1}Z\right\|_F^2 - \frac{1}{2\sigma}\|Z\|_F^2$$

Subproblem-L (singular value soft-thresholding):
1. Compute SVD: $T^{(k)} = X - S^{(k)} - \sigma^{-1}Z^{(k)} = U^{(k)}\,\mathrm{Diag}(d^{(k)})\,(V^{(k)})^T$
2. Soft-threshold singular values: $\gamma^{(k)} = \mathcal{S}_{1/\sigma}(d^{(k)})$
3. Update: $L^{(k+1)} = U^{(k)}\,\mathrm{Diag}(\gamma^{(k)})\,(V^{(k)})^T$

Subproblem-S (element-wise soft-thresholding):

$$S^{(k+1)} = \mathcal{S}_{\lambda/\sigma}\left(X - L^{(k+1)} - \sigma^{-1}Z^{(k)}\right)$$

Multiplier update:

$$Z^{(k+1)} = Z^{(k)} + \tau\sigma\left(L^{(k+1)} + S^{(k+1)} - X\right)$$

5. ADMM for Lasso

Lasso: $\min_{\beta \in \mathbb{R}^p} \frac{1}{2}\|X\beta - Y\|^2 + \lambda|\beta|_1, \quad X \in \mathbb{R}^{n \times p},\ Y \in \mathbb{R}^n$
Primal ADMM — use when $p < n$
Introduce slack $u = \beta$:

$$\min_{\beta, u} \frac{1}{2}\|X\beta - Y\|^2 + \lambda\|u\|_1 \quad \text{s.t.} \quad \beta - u = 0$$

Iterations:

$$\beta^{(k+1)} = \left(\sigma I + X^TX\right)^{-1}\left(X^TY + \sigma u^{(k)} - \xi^{(k)}\right) \quad [p \times p \text{ linear system}]$$
$$u^{(k+1)} = \mathcal{S}_{\lambda/\sigma}\left(\beta^{(k+1)} + \sigma^{-1}\xi^{(k)}\right)$$
$$\xi^{(k+1)} = \xi^{(k)} + \tau\sigma\left(\beta^{(k+1)} - u^{(k+1)}\right)$$

Dual ADMM — use when $n < p$
Dual problem:

$$\min_{y, v} \frac{1}{2}\|y\|^2 - \langle y, Y \rangle + \delta_{B_\lambda}(v) \quad \text{s.t.} \quad X^Ty + v = 0$$

Iterations:

$$y^{(k+1)} = \left(I + \sigma XX^T\right)^{-1}\left(Y - X\left(\beta^{(k)} + \sigma v^{(k)}\right)\right) \quad [n \times n \text{ linear system}]$$
$$v^{(k+1)} = \Pi_{B_\lambda}\left(-X^Ty^{(k+1)} - \sigma^{-1}\beta^{(k)}\right)$$
$$\beta^{(k+1)} = \beta^{(k)} + \tau\sigma\left(X^Ty^{(k+1)} + v^{(k+1)}\right)$$

where $B_\lambda = \{v \mid \|v\|_\infty \le \lambda\}$ and $\Pi_{B_\lambda}(s_i) = \mathrm{clamp}(s_i, -\lambda, \lambda)$.

6. Key Operators

Operator	Formula	Used in
Soft-thresholding $\mathcal{S}\tau(x)_i$	$\mathrm{sign}(x_i) \cdot \max(\|x_i\| - \tau, 0)$	ℓ_1 proximal
SVD soft-threshold	Apply $\mathcal{S}_{1/\sigma}$ to singular values	Nuclear norm proximal
Projection onto B_λ	Clamp each entry to $[-\lambda, \lambda]$	Dual Lasso
Projection onto set C	$\Pi_C(v) = \arg\min_{z \in C}\|z - v\|$	Constrained problems

7. Practical Notes

- σ **too large** ⇒ insufficient minimization of $f + g$
- σ **too small** ⇒ insufficient feasibility enforcement
- Common step lengths: $\tau = 1$ or $\tau = 1.618 \approx \frac{1+\sqrt{5}}{2}$
- Pre-compute and cache $(\sigma I + X^TX)^{-1}$ or $(I + \sigma XX^T)^{-1}$ before iterating
- Choose **primal vs. dual ADMM** based on problem dimension:
  - $p < n$ ⇒ primal ADMM ($p \times p$ system)
  - $n < p$ ⇒ dual ADMM ($n \times n$ system)

Lecture 10: Gaussian Graphical Models

1. Conditional Independence

Definition: $X \perp\!\!\!\perp Y \mid Z$ iff $p(x, y \mid z) = p(x \mid z)p(y \mid z)$
Factorization: If $X \perp\!\!\!\perp Y \mid Z$, then $p(x, y, z) = g(x, z)\,h(y, z)$

2. Undirected Graph

- Graph $G = (V, E)$, vertex set $V = \{1, \ldots, p\}$, edge set $E \subset V \times V$
- Undirected: $(s,t) \in E \iff (t,s) \in E$; no self-loops

Markov Property: x is Markov w.r.t. G if for every cut set S separating A and B:

$$x_A \perp\!\!\!\perp x_B \mid x_S$$

3. Multivariate Gaussian

$$f(x) = (2\pi)^{-p/2}\det(\Theta)^{1/2}\exp\left(-\frac{1}{2}x^T\Theta x\right), \quad x \sim \mathcal{N}(0, \Sigma)$$

- **Precision matrix:** $\Theta = \Sigma^{-1} \in S_{++}^p$
- $\det(\Sigma) = \prod_j \lambda_j(\Sigma), \quad \det(\Theta) = 1/\det(\Sigma)$

4. Gaussian Graphical Model (GGM)

Key fact: For $x \sim \mathcal{N}(0, \Sigma)$:

$$x_s \perp\!\!\!\perp x_t \mid x_{V \setminus \{s,t\}} \iff \Theta_{st} = 0$$

Edge set: $E = \{(s,t) \mid \Theta_{st} \neq 0,\ s < t\}$

Zero off-diagonal entry in $\Theta \longleftrightarrow$ no edge $\longleftrightarrow$ conditional independence

Steps to draw the graph:
1. Compute $\Theta = \Sigma^{-1}$
2. Find all (s,t) with $\Theta_{st} \neq 0$, $s < t$ → these are the edges
3. Plot

5. Maximum Likelihood Estimation (MLE)

Given n i.i.d. samples, the **sample covariance matrix** is:

$$S = \frac{1}{n}\sum_{i=1}^{n} x^{(i)}(x^{(i)})^T$$

MLE problem:

$$\min_{\Theta \in S_{++}^p} -\log\det(\Theta) + \langle S, \Theta \rangle$$

- MLE solution (when it exists): $\hat{\Theta} = S^{-1}$
- Solution **fails** when $n < p$ (S is rank-deficient)

Log-det function: $h(\Theta) = -\log\det(\Theta)$

- Convex on S_{++}^p
- Gradient: $\nabla h(\Theta) = -\Theta^{-1}$

6. Graphical Lasso

Adds ℓ_1 penalty on off-diagonal entries to encourage sparsity:

$$\min_{\Theta \in S_{++}^p} -\log\det(\Theta) + \langle S, \Theta \rangle + \lambda\|\Theta\|_{1,\mathrm{off}}$$

where $\|\Theta\|_{1,\mathrm{off}} = \sum_{s \neq t}|\Theta_{st}|$

7. ADMM for Graphical Lasso

Reformulate with splitting variable Y, Lagrange multiplier Z:

Step	Update
Θ-update	$T = Y - \sigma^{-1}(S + Z)$; eigen-decomp $T = Q\,\mathrm{Diag}(\rho)\,Q^T$; then $\gamma_j = \frac{1}{2}\left(\rho_j + \sqrt{\rho_j^2 + 4/\sigma}\right)$; $\Theta \leftarrow Q\,\mathrm{Diag}(\gamma)\,Q^T$
Y-update	$Y \leftarrow S_{\lambda/\sigma}^{\mathrm{off}}(\Theta + \sigma^{-1}Z)$ — soft-threshold **off-diagonal** only
Z-update	$Z \leftarrow Z + \tau\sigma(\Theta - Y)$, with $0 < \tau < \frac{1+\sqrt{5}}{2}$

Soft-threshold operator: $S_\alpha(x) = \mathrm{sign}(x)\max(|x| - \alpha, 0)$
Proximal map of log-det:

$$P_{\frac{1}{\sigma}h}(Y) = Q\,\mathrm{Diag}(\gamma)\,Q^T, \quad \gamma_j = \frac{1}{2}\left(\rho_j + \sqrt{\rho_j^2 + 4/\sigma}\right)$$

8. Neighbourhood Selection

Idea: Regress each variable on all others using Lasso.
For vertex j, solve:

$$\hat{\beta}^j = \arg\min_{\beta \in \mathbb{R}^{p-1}} \frac{1}{2}\|X_{\cdot,-j} - \beta \cdot X_{\cdot,j}\|^2 + \lambda|\beta|_1$$

Neighbourhood: $\mathcal{N}(j) = \{t \mid \beta_t^j \neq 0\}$
Graph construction:
- **AND rule:** edge (s,t) iff $s \in \mathcal{N}(t)$ **and** $t \in \mathcal{N}(s)$
- **OR rule:** edge (s,t) iff $s \in \mathcal{N}(t)$ **or** $t \in \mathcal{N}(s)$

9. Quick Reference

Symbol	Meaning
Σ	Covariance matrix
$\Theta = \Sigma^{-1}$	Precision (inverse covariance) matrix
S	Sample covariance matrix
λ	Regularisation parameter
σ	ADMM step size
S_{++}^p	Set of $p \times p$ positive definite matrices

Lecture 11: Second Order Methods

1. Rates of Convergence

Type	Condition	Example
Q-linear	$\frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|} \le r \in (0,1)$	$1 + 0.8^k$
Q-superlinear	$\lim_{k\to\infty} \frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|} = 0$	$1 + k^{-k}$
Q-quadratic	$\frac{\|x^{(k+1)} - x^*\|}{\|x^{(k)} - x^*\|^2} \le M$ for some $M > 0$	$1 + (0.8)^{2^k}$

Hierarchy: Q-quadratic > Q-superlinear > Q-linear (eventually faster)
Key facts:
- Q-quadratic ⇒ Q-superlinear ⇒ Q-linear
- $1/k$ is **not** Q-linearly convergent; $1/k!$ is Q-superlinear but **not** Q-quadratic

2. General Framework

For $\min_x f(x)$, differentiable:

$$x^{(k+1)} = x^{(k)} - \alpha_k P_k \nabla f(x^{(k)})$$

P_k	Method
I	Gradient (steepest) descent
$[H_f(x^{(k)})]^{-1}$	Pure Newton's method
$[H_f(x^{(k)}) + \tau_k I]^{-1}$	Modified Newton
$H_k \approx [H_f(x^{(k)})]^{-1}$	Quasi-Newton (BFGS)

3. Pure Newton's Method

Derivation: Minimize 2nd-order Taylor approximation of f.
Newton direction:

$$p^{(k)} = -[H_f(x^{(k)})]^{-1}\nabla f(x^{(k)})$$

equivalently, solve the **linear system**:

$$H_f(x^{(k)})p = -\nabla f(x^{(k)})$$

Update: $x^{(k+1)} = x^{(k)} + p^{(k)}$ (step size = 1)
Descent direction condition: $p^{(k)}$ is a descent direction iff $H_f(x^{(k)}) \succ 0$:

$$\nabla f(x^{(k)})^T p^{(k)} = -(p^{(k)})^T H_f(x^{(k)})p^{(k)} < 0$$

Convergence: Q-quadratic (locally, near x^*), given:
1. f twice differentiable
2. $H_f(\cdot)$ locally Lipschitz at x^*: $|H_f(x^*) - H_f(x)\| \le L|x^* - x\|$
3. Sufficient conditions (SOSC) at x^*

Limitations: May diverge from remote starting points; requires Hessian ($O(n^3)$ to solve).

4. Practical Newton's Method

4a. Line Search Newton with Modification

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}, \quad p^{(k)} = -[H_f(x^{(k)}) + \tau_k I]^{-1}\nabla f(x^{(k)})$$

Hessian modification: If $H_f(x^{(k)}) \nsucc 0$, choose $\tau_k > -\lambda_{\min}(H_f(x^{(k)}))$ so $B_k = H_f(x^{(k)}) + \tau_k I \succ 0$.
- If τ_k too large: $p^{(k)} \approx -\tau_k^{-1}\nabla f(x^{(k)})$ (degenerates to slow gradient descent)

Line search (Armijo backtracking):

$$f(x^{(k)} + \alpha p^{(k)}) \le f(x^{(k)}) + c_1\alpha\nabla f(x^{(k)})^T p^{(k)}$$

Start with $\alpha = 1$, reduce by $\alpha \leftarrow \rho\alpha$ until satisfied.
Convergence rate:
- If $H_f(x^*) \succ 0$: $\tau_k \to 0$, reduces to pure Newton → **Q-quadratic**
- Otherwise: may be only **linear**

4b. BFGS (Quasi-Newton)

Approximates $H_k \approx [H_f(x^{(k)})]^{-1}$ without computing the true Hessian.
Secant equation (the key constraint on $B_{k+1} \approx H_f$):

$$B_{k+1}s_k = y_k$$

where $s_k = x^{(k+1)} - x^{(k)}, \quad y_k = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)})$
Curvature condition (required for $B_{k+1} \succ 0$): $y_k^T s_k > 0$
BFGS update formula (updates inverse Hessian approximation H_k):

$$H_{k+1} = (I - \rho_k s_k y_k^T)H_k(I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \rho_k = \frac{1}{y_k^T s_k}$$

Equivalent B_k **update (rank-2):**

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

Per-iteration cost: $O(n^2)$. **Convergence:** Q-superlinear.

5. Trust-Region Method

Model function:

$$m_k(p) = f(x^{(k)}) + \nabla f(x^{(k)})^T p + \frac{1}{2}p^T B_k p$$

Subproblem: $\min p\ m_k(p) \quad \text{s.t.} \quad \|p\| \le \Delta_k$
Ratio for adapting radius:

$$\rho_k = \frac{f(x^{(k)}) - f(x^{(k)} + p^{(k)})}{m_k(0) - m_k(p^{(k)})} = \frac{\text{actual reduction}}{\text{predicted reduction}}$$

ρ_k	Action
$< 1/4$	Shrink: $\Delta_{k+1} = \frac{1}{4}\Delta_k$, reject step
$> 3/4$ and $\|p^{(k)}\| = \Delta_k$	Expand: $\Delta_{k+1} = \min(2\Delta_k, \bar{\Delta})$
otherwise	Keep: $\Delta_{k+1} = \Delta_k$
$> \eta$	Accept step: $x^{(k+1)} = x^{(k)} + p^{(k)}$

Full step (when $B_k \succ 0$ and $\|B_k^{-1}\nabla f(x^{(k)})\| \le \Delta_k$):

$$p^{(k)} = -B_k^{-1}\nabla f(x^{(k)})$$

6. Proximal Newton (Non-smooth g)

For $\min_x f(x) + g(x)$ (f smooth, g non-smooth):

$$x^{(k+1)} = \arg\min_z \left\{ \nabla f(x^{(k)})^T(x - x^{(k)}) + \frac{1}{2}(x - x^{(k)})^T B_k(x - x^{(k)}) + g(x) \right\}$$

Quick Reference: Convergence Rates

Method	Rate
Gradient descent	Q-linear
Newton's (pure, local)	Q-quadratic
Newton's (modified + line search, at x^* with $H \succ 0$)	Q-quadratic
Newton's (at x^* with $H \nsucc 0$)	Q-linear
BFGS	Q-superlinear
Trust-region Newton (at SOSC point)	Q-superlinear

Lecture 12: Clustering

1. Core Idea

	Supervised	Unsupervised
Labels	Given (Y)	None
Goal	Predict labels	Find hidden patterns
Examples	SVM, linear regression	K-means, hierarchical

Clustering groups data so that:
- **Intra-cluster distance** is **small** (similar points together)
- **Inter-cluster distance** is **large** (different groups far apart)

2. Distance Between Two Objects

$$D(a, b) = \|a - b\|_2 = \sqrt{\sum_{i=1}^{p}(a_i - b_i)^2}$$

Other options: $\|a - b\|_1$, $\|a - b\|_\infty$, cosine similarity.

3. Distance Between Two Clusters

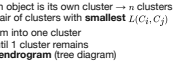
Linkage	Formula	Description
Single	$\min_{a \in C_1, b \in C_2} D(a, b)$	Shortest pairwise distance
Complete	$\max_{a \in C_1, b \in C_2} D(a, b)$	Longest pairwise distance
Average ⭐	$\frac{1}{\|C_1\|\|C_2\|}\sum_{a \in C_1, b \in C_2} D(a, b)$	Average of all pairwise distances

⭐ Average linkage is the most widely used.

4. Hierarchical Clustering (Bottom-Up / Agglomerative)

Algorithm:
1. Start: each object is its own cluster → n clusters
2. Find the pair of clusters with **smallest** $L(C_i, C_j)$
3. Merge them into one cluster
4. Repeat until 1 cluster remains

Output: A dendrogram (tree diagram)



- **Low fusion height** → very similar objects
- **High fusion height** → dissimilar objects
- **Horizontal position** along the x-axis means **nothing** about similarity

Reading the Dendrogram

Q	How to answer
Are X and Y similar?	Check how low they fuse — lower = more similar
How different are two clusters?	Read the **height** on the vertical axis
Is A more similar to B than to C?	**Cannot** determine from dendrogram alone

Identifying Clusters: Horizontal Cut
Make a horizontal cut at height $y = a$:
- Number of vertical lines crossed = **number of clusters**

Choosing the best cut: pick the cut with the **largest vertical range** (most room to move up/down without hitting a horizontal branch).

```
Cut at y=6  →  3 clusters: {C,G,G,A,D}, {B}, {E,F}   (range 1.91) ✓
Cut at y=3.4 →  5 clusters: {C,G}, {A,D}, {B}, {E}, {F}  (range 0.44)
```

5. K-Means Clustering

Objective: minimise total intra-cluster distance

$$C_1 \ldots C_K \sum_{k=1}^{K} \frac{1}{|C_k|} \sum_{i,j \in C_k} \|x_i - x_j\|^2$$

Algorithm:

```
Initialise: place K centroids μ₁, ..., μ_K at random

Repeat until assignments don't change:
  (1) Assign each x_i to nearest centroid:
        argmin_k  D(x_i, μ_k)

  (2) Recompute centroid of each cluster:
        μ_k = (1/|C_k|) Σ_{i∈C_k} x_i
```

Choosing K (Elbow Method)
Plot the objective function vs K. Look for the **"elbow"** — the point of most abrupt decrease.

```
Objective
  |  *
  |   *
  |    *
  |     * * * * *
  +—1——2——3——4——5— K
                ↑
          elbow → K=2 or K=3
```

6. Hierarchical vs K-Means Summary

	Hierarchical	K-Means
K specified?	No (cut dendrogram after)	Yes, upfront
Output	Dendrogram	Flat cluster assignments
Sensitive to init?	No	Yes — run multiple times
Key choice	Linkage type + cut height	K, initialisation
Overlapping clusters?	No	No

7. Common Pitfalls & Practical Notes

- **K-means:** results depend on random initialisation → run multiple times, pick lowest objective value
- **Hierarchical:** choice of linkage affects results significantly
- **Both:** no universally "correct" number of clusters — use elbow plot or dendrogram gap heuristic
- **Horizontal position** in a dendrogram carries **no similarity information** — only height matters